GUILHOT Benjamin
VAN DOORNIK Levi

Advanced Topic Reflection - BenLeViRPG

Testing with Junit

Why we chose this topic

We chose this topic because we knew that we wanted to make an RPG game so we can't create something with loads of bugs. We had to test the maths of the game: Is the crit rate well implemented? The xp generation? The coin generation? Etc… That's why we needed a tool that provides us a security measure against regressions.

What we learned

We learned how to add a dependency in Java, how to use such a dependency, how to run tests, etc… But the most important was to switch to a test driven development method. We decided that each core file should have a test file and that, before each commit, we should write the tests about what we've just developed. The built-in methods like assertTrue and assertEquals were also new for us so we learned where to find them and how to use them.

How we applied it

We applied it by doing 10 test files which represents 57 tests. We focused on the most important mechanics of the game (attack, deal damage, receive damage, crit, type, ultimate, …) and we tested them by trying to find the edge cases, the main one and we computed the result on paper and we used the unit tests to compare the output of the game to our computation.

What we learned from the implementation

The main thing we learned was that we didn't think about edge cases at all without the tests so the bugs appear. For example, we didn't think that it'd be possible to win a negative amount of coins, that it's possible to deal -2 damage, etc… The implementation was quite easy when you got used to it, but at the beginning we didn't know where to start. In the future, it'll be easier for us to implement it again.

UX & Game Design

Why we chose this topic

When playing a game, it is important to have a good experience. How intuitive does the game feel and what might be confusing? We chose this topic to learn how to make these impactful changes, and see what difficulties may arise.

What we learned

We learned that balancing the game is more difficult than we thought at first. XP, coins, levels, damage, it all needs to be tuned so fights don't turn out to be way too boring, or way too difficult. We also learned that it's important to show what is going on, how far you are in the game and how much progress you've made.

How we applied it

We designed a system with four types (Normal, Fire, Water, Thunder), forming a rock-paper-scissors type of system with strengths and weaknesses. We added a character selection screen that looks more friendly than the default Swing look. Also, to quickly see what type you and the enemy is, we changed the color to match the type, so we have Fire, Water, and Thunder. We added character sprites for each character and enemy, also very distinct, so it is clear what you are fighting with and against. For each hero, we added custom stats and a special attack, to spice the gameplay up even more. Finally, we added a coin and shop system, so you can upgrade your characters, and win even if you're at a type disadvantage.

What we learned from the implementation

The most important lesson was that balance problems only appear when you play the game. Early in development, the Warrior had an ATK of 5 against a Fire-type enemy with a type advantage, which reduced the effective damage to 1 per hit, making the fight nearly unwinnable. On the UI side, we discovered that Swing's default button rendering conflicts with custom background colors on some platforms unless setOpaque(true) and setBorderPainted(false) are set explicitly. These small details took time to get right but had a large impact on how the final application looks.